Chapter 1

Other Graph Algorithms and Proofs

This chapter is an overview of other graph algorithms, proofs, and concepts that are important to know. We will first cover adjacency matrices, then Prim's algorithm for minimum spanning trees, isomorphism, and topological sorting for directed acyclic graphs (DAGs). A lot of these are very advanced graph theory topics, so you won't be tested on too heavily of these topics. Many of these appear on AP/IB exams, or in advanced college level CS classes.

1.1 Adjacency Matrices

Recall that we can represent graphs as lists of data called adjacency lists. In code, this could look like the following:

- A: [B, C]
- B: [A, D]
- C: [A]
- D: [B]

This represents the following graph

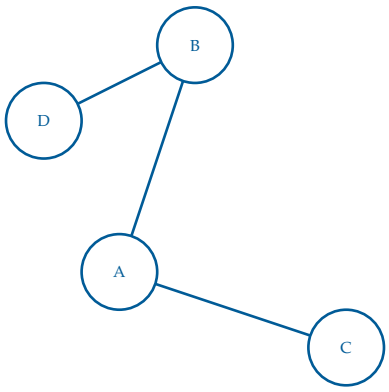


Figure 1.1: A simple (undirected unweighted) graph used to make an adjacency list and adjacency matrix.

We could also represent the same data as a *matrix*, which we have covered in linear algebra. The *adjacency matrix* is 2D array (as compared to an adjacency list, which consists of multiple 1D arrays) where `matrix[i][j]` indicates whether an edge exists from vertex i to vertex j. Commonly, `matrix[i][j]` $= 1$ if and only if the edge from vertex i to vertex j exists, else `matrix[i][j]` $= 0$.[1]The above graph in Figure 1.1 can be represented as:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

For *undirected* graphs, the adjacency matrix is symmetric, and thusly $n \times n$, meaning that the transpose of matrix is the same matrix!

$$A = A^T$$

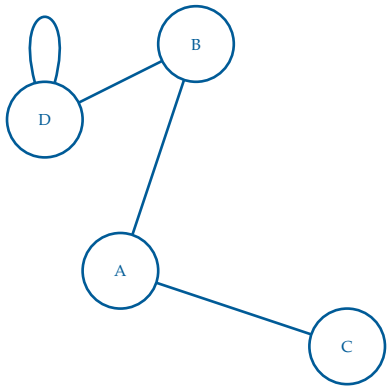


Figure 1.2: A simple graph with an added loop on vertex D.

By convention, a diagonal entry such as `matrix[k][k]`, where $(k \in \mathbb{Z})$, is non-zero only if there is a loop from a vertex to itself. The value of the entry represents the number of such loops. In this case, there are two edges from vertex (D) to itself in Figure 1.2, so the corresponding diagonal entry is (2).

For *directed* graphs, the adjacency matrix on appears a bit different. A *directed adjacency matrix* represents a directed graph, where edges have a direction (arrows). When $A_{ij} = 1$, then an edge exists from vertex i to vertex j.

[1]If the edges are weighted, the matrix[i][j] is equal to the weight, or 0 if the edge does not exist.

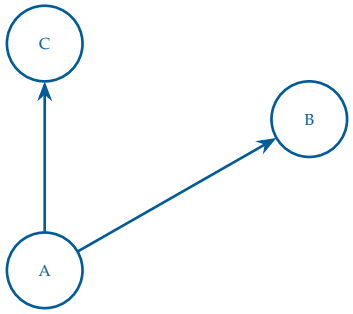


Figure 1.3: A simple directed graph used to make an adjacency matrix.

The resulting adjacency matrix is

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Notice how, now, $A \neq A^T$, because the edges are directional.

Exercise 1 Adjacency Matrix Exercise

Working Space

The following is an adjacency matrix for a weighted (undirected) graph.

	A	B	C	D	E	F	G
A	0	4	2	0	0	0	0
B	4	0	1	5	0	0	0
C	2	1	0	8	10	0	0
D	0	5	8	0	2	6	0
E	0	0	10	2	0	3	7
F	0	0	0	6	3	0	4
G	0	0	0	0	7	4	0

Draw the corresponding graph with weighted edges.

Answer on Page 17

1.2 Prim's Algorithm and Kruskal's Algorithm for Minimum Spanning Trees

Prim's Algorithm is minimum spanning tree

FIXME https://github.com/KontinuaFoundation/sequence/issues/315

1.3 A* (A-star) Search Algorithm

The last shortest path we will discuss in this chapter is the *A-star Algorithm*, more commonly represented as *A* Algorithm*. In essence, it is an extension of Dijkstra's Algorithm in combination with a *priority-based heuristic function*. A heuristic is an introduced estimated cost from any node to the goal.

The key behind this implementation is the following function

$$f(n) = g(n) + h(n)$$

where

- $g(n)$: the cost of the path from the start node (usually S) to the current node n
- $h(n)$: the *heuristic* function that estimates the remaining cost from n to the goal (usually E)
- $f(n) = g(n) + h(n)$: the estimated total cost of a path from S to E passing through node n

1.3.1 The Path Cost $g(n)$

$g(n)$ remains to be the same value that we used in Dijkstra's and other shortest path algorithms.

When moving from a node u to an adjacent node v, the path cost is *updated* as

$$g(v) = g(u) + c(u, v)$$

where $c(u, v)$ is the cost of traversing the edge from u to v. As we discussed in the chapter on Dijkstra's, we have a decision to make upon each visit to a node: Each time we visit a node, we have a decision to make:

$$\text{if } dist[u] + w(u, v) < dist[v] \text{ then update } dist[v]$$

1.3.2 The Heuristic Function $h(n)$

The heuristic $h(n)$ function depends on the problem domain. Its purpose is to estimate the remaining cost from node n to the goal. While this is a difficult task, we can use basic algebra to make an informed guess.

For example, in a map-based pathfinding problem, $h(n)$ may be the straight-line distance, or *Euclidean distance* between node n and the goal. Since the actual path cannot be shorter than the Euclidean distance, this provides a reasonable estimate of the remaining cost. If a nodes location is represented on a 2D plane (such as a map or graph) (x_n, y_n) and the goal location as (x_g, y_g), the Euclidean Heuristic

$$h(n) = \sqrt{(x_g - x_n)^2 + (y_g - y_n)^2}$$

Alternatively, many 2D games use a similar either *taxicab distance* (typically for 4-way movement),

$$h(n) = |x_g - x_n| + |y_g - y_n|$$

or *Chebyshev distance* (typically for 8-way movement in a game)

$$d = \max(|x_2 - x_1|,\ |y_2 - y_1|)$$

which takes the maximum of the horizontal and vertical movements.

Sometimes, graphs without direct spatial meaning become more difficult to work with because there is no obvious notion of distance between two nodes.

For general graphs, such as social networks, communication networks, or dependency graphs, the heuristic must be derived from problem-specific knowledge. For example, a social network application might use information such as mutual connections, community membership, or other metadata to estimate how *close* a user is to a target user.

In many cases, however, no suitable heuristic is known. In these situations, the heuristic is often defined as

$$h(n) = 0$$

for all nodes n. Substituting this into the A* evaluation function gives

$$f(n) = g(n)$$

which causes A* to reduce to to Dijkstra's algorithm. While this guarantees an optimal path, it loses the efficiency gains that a good heuristic can provide.

There is an important property that the Heuristic Function must satisfy:

$$h(n) \leq \text{true remaining cost}$$

When an admissible heuristic is used (such that $h(n) \leq$ true remaining cost), A* is guaranteed to find an optimal path if one exists.

1.3.3 The Total Estimated Distance $f(n)$ and Priority Queues

The purpose of the function

$$f(n) = g(n) + h(n)$$

is to assign a priority to each node during the search. Recall from our discussion of priority queues that each element is associated with a priority value, and the element with the highest (or lowest) priority is removed first.

In A*, the priority of a node is its estimated total distance $f(n)$. Nodes with smaller values of $f(n)$ are considered more promising because they represent paths that appear likely to reach the goal with a lower total cost.

As with Dijkstra's algorithm, A* maintains a priority queue containing nodes that have been discovered but not yet fully explored. However, instead of prioritizing nodes solely by their known distance from the start node, A* prioritizes nodes according to their estimated total distance

$$\text{priority}(n) = f(n)$$

At each step, the node with the smallest $f(n)$ value is removed from the priority queue and explored. When a neighboring node is discovered, its path cost $g(n)$ is updated, a new heuristic estimate $h(n)$ is calculated, and a new value of $f(n)$ is computed.

There are some good references about this in your digital resources, as well as a fun Minecraft-based explanation here.

1.4 Injection, Surjection, Bijection, and Isomorphism

Let's talk about Graph Isomorphism. To do this, we must first introduce the terms of *injection*, *surjection*, and *bijection*. These are types of functions in set theory as well as mathematics.

Definition 1 (Injection)**.** *A function* $f : A \to B$ *is injective if different inputs always produce different outputs. Importantly, this means no two integers map to the same output.*

Definition 2 (Surjection)**.** *A function* $f : A \to B$ *is surjective if every element of B is hit by at least one element of A. In essence, all of the elements of B are covered.*

Definition 3 (Bijection)**.** *A function* $f : A \to B$ *is bijective if it is both injective and surjective. Ultimately, the function creates no collisions and hits all functions of B.*

Exercise 2 Bijective, Surjective, Injective Exercise

Working Space

Determine whether each function is injective, surjective, bijective, or none of these.

1. $f : \{1, 2, 3\} \to \{a, b, c, d\}$ defined by

   $$f(1) = a, \quad f(2) = b, \quad f(3) = c.$$

2. $g : \{1, 2, 3, 4\} \to \{a, b, c\}$ defined by

   $$g(1) = a, \quad g(2) = b, \quad g(3) = c, \quad g(4) = a.$$

3. $h : \{1, 2, 3\} \to \{a, b, c\}$ defined by

   $$h(1) = b, \quad h(2) = c, \quad h(3) = a.$$

Answer on Page 17

Now, let's talk about *Isomorphism*. A *graph isomorphism* is a bijection between the vertex sets of two graphs that preserves adjacency. Thus, an isomorphism not only matches vertices one-to-one, but also preserves the structure of the graph.

Definition 4 (Graph Isomorphism)**.** *Let* $G = (V_G, E_G)$ *and* $H = (V_H, E_H)$ *be graphs. An isomorphism from* G *to* H *is a bijection*

$$f : V_G \to V_H$$

such that for all vertices $u, v \in V_G$*,*

$$\{u, v\} \in E_G \iff \{f(u), f(v)\} \in E_H.$$

If such a function exists, we say that G *and* H *are isomorphic and write*

$$G \cong H.$$

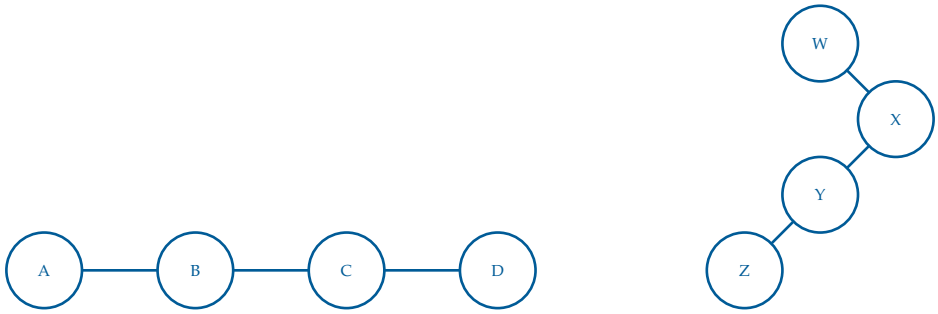


Figure 1.4: Two graphs that are isomorphic. Graph G (left) and Graph H (right).

We are able to say that these graphs (Figure 1.4) are isomorphic as one vertex from G can be paired with one vertex from H, while maintaining the same edge connections.

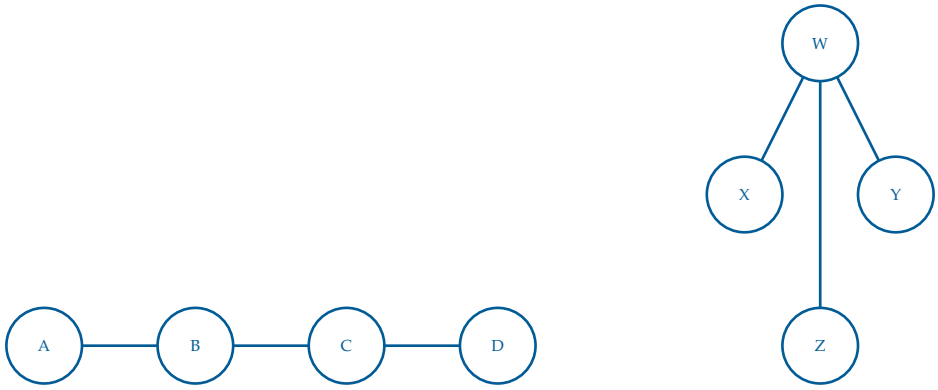


Figure 1.5: Graph X and Y; not isomorphic.

In Figure 1.5, we can see the vertices are not isomorphic without adding or subtracting edges. For example, Graph X does not have a vertex with 3 edges, unlike Graph Y.

If any of these differ, the graphs are not isomorphic:

- Number of vertices
- Number of edges
- Degree sequence (if degree sequences are intact, it does not guarantee isomorphism)
- Number of connected components
- Number of cycles of a given length
- Chromatic number

Example.

A common example that you will not be tested on but is important to know is the following. We may need to introduce a definition first.

Definition 5 (Clique)**.** *A clique is a set of vertices in a graph such that every pair of distinct vertices is connected by an edge.*

Definition 6 (Clique Subgraph)**.** *A* k*-clique is exactly a subgraph isomorphic to the complete graph* K_k*. A 3-clique would be a clique of size 3, called* K_3*.*

Now, given two graphs G and H, how can we determine G contain a subgraph isomorphic to H, a graph of smaller to equal size to G?

This may be easy if G and H are pretty small, but the math becomes infinitely harder as the number of vertices and edges go up.

We can simplify this by transforming the inputs of a clique problem to inputs of the subgraph isomorphism problem.

The clique problem gives us a graph M and a threshold k. The output of this is whether M contains a clique of size at least k.

To transform this into an instance of the subgraph isomorphism problem, construct a smaller subgraph $H = K_k$, the complete graph on k vertices, and let $G = M$, the larger graph.

We now ask the subgraph isomorphism question:

> Does G contain a subgraph isomorphic to H?

Since H is the complete graph K_k, a subgraph of G isomorphic to H must contain k vertices where every pair of vertices is connected by an edge. By definition, this is exactly a k-clique.

Therefore,

$$M \text{ contains a } k\text{-clique} \iff G \text{ contains a subgraph isomorphic to } K_k.$$

This shows that every instance of the Clique problem can be transformed into an instance of the Subgraph Isomorphism problem. We call this an *algorithmic reduction*.[2]

[2] Algorithmic Reductions are the basis of a set of problems called P, NP, and NP-Complete. See this video for more.

1.5 Travelling Salesperson Problem and Nearest Neighbour

FIXME https://github.com/KontinuaFoundation/sequence/issues/312

1.6 Topological Sort and DAGs

Another common use of graph algorithms is sorting them in a meaningful order. Specifically, we can sort directed graphs that tells us exactly what order vertices need to be "visited" or operated on. We call this a *Topological Sorting* of the graph. A *topological order* is a linear ordering of the vertices in a graph such that for every directed edge $u \to v$ from vertex u to vertex v, vertex u comes before vertex v in the ordering. The most common usage for this is scheduling jobs, queueing tasks with dependencies, or creating tentative project schedules.

To do this, we are going to use Python's `Graphlib`, and its `TopologicalSorter` module.

In the `graphlib` module, graphs are defined in the following format:

```
{
  node: {predecessors}
}
```

A basic example of this is the following

```
{
  "C": {"A", "B"}
}
```

Which says that A and B need to come before C, such that one possible ordering is

$$A \to B \to C$$

Now suppose we alter the graph format as follows:

```
{
  "C": {"A", "B"},
  "A": {"Z"},
  "B": {"Z"},
  "Z": {"C"}
}
```

Here,

- C depends on A and B
- A and B depend on Z
- Z depends on C

There is no way to determine a *starting point*. Now we have an ordering that contains a cycle. This implies that C must come before itself, which is impossible in a linear ordering. Therefore no topological ordering can exist.

Definition 7 (Cycle)**.** *A cycle is a valid walk* $(v_1, v_2, \ldots, v_n)$ *where the only repeated vertex is* $v_n = v_1$*, meaning the last vertex equals the first vertex.*

So, we cannot do a topological sort on a graph that contains a cycle. This provides us with a theorem:

Topological Sorting and DAGs

A directed graph admits a topological ordering if and only if it is a *Directed Acyclic Graph* (DAG). That is, the directed graph may not contain any cycles (acyclic).

In fact, the python graphlib library will not produce a valid Topological Sort if a cycle exists. Running the following code

```
from graphlib import TopologicalSorter
graph = {
  "C": {"A", "B"},
  "A": {"Z"},
  "B": {"Z"},
  "Z": {"C"}
}
list(TopologicalSorter(graph).static_order())
```

will produce the following error

```
graphlib.CycleError: ('nodes are in a cycle', ['C', 'Z', 'A', 'C'])
```

The exception reports one cycle that prevents the ordering from being constructed. In this case, C → Z → A → C forms a directed cycle.

We can also introduce tasks that can be done in parallel. Create the following file `parallel_dag.py`

```
from graphlib import TopologicalSorter
tasks = {
    "Compile": set(),
```

```
    "Build Frontend": {"Compile"},
    "Build Backend": {"Compile"},
    "Test": {"Build Frontend", "Build Backend"},
    "Deploy": {"Test"},
}from graphlib import TopologicalSorter
tasks = {
    "Compile": set(),
    "Build Frontend": {"Compile"},
    "Build Backend": {"Compile"},
    "Test": {"Build Frontend", "Build Backend"},
    "Deploy": {"Test"},
}
from graphlib import TopologicalSorter

ts = TopologicalSorter(tasks)
ts.prepare()

while ts.is_active():
    ready = ts.get_ready()
    print(ready)

    for task in ready:
        ts.done(task)

```

The 5 nodes here are tasks for a basic software project. The ordering is as follows:

$$\text{Compile} \rightarrow \text{Build Frontend and Build Backend} \rightarrow \text{Test} \rightarrow \text{Deploy}$$

However notice that *Build Frontend* and *Build Backend* are not dependent on each other, so they could be done in *parallel*. The *Test* task cannot begin until both builds have finished. Running this code outputs the following

```
('Compile',)
('Build Frontend', 'Build Backend')
('Test',)
('Deploy',)
```

showing us that our sort outputs which ones can happen at the same time. In software development, many tasks have dependencies but can still be executed in parallel when those dependencies are satisfied. Topological sorting helps determine both the required execution order and which tasks may be performed concurrently (this is similar to running circuits in parallel!).

1.7 Proofs

This section will be a set of proofs or algorithm design problems for graph-related concepts that are important to know. At the basis level, these are just practice problems for graph understand, but can be applied in many real world scenarios.

1.7.1 Example 1: Bipartite

Bipartite Graph

A *Bipartite Graph* is a graph $G = (V, E)$ such that V can be partitioned into two sets $(V = V_1 \cup V_2)$ and $(V_1 \cap V_2 = empty)$ such that there are no edges between vertices in the same set. FIXME Diagram

Theorem 1. *An undirected graph* G *is bipartite if and only if it contains no cycles of odd length.*

Exercise 3 Algorithm for Determining Bipartite

Design pseudocode for an algorithm that determines whether or not an undirected graph is bipartite. *Hint: use the theorem.*

Hint 2: This function may be useful

`FindOdd`(G, e)

Require: G, a graph on which DFS has been run; $e = (x, y)$, a back edge
Ensure: **true** if e creates a cycle of odd length, **false** otherwise

```
1: if pre(x)−post(y) ≡ 0 (mod 2) then
2:     return true
3: else
4:     return false
5: end if
```

Working Space

Answer on Page 18

While you don't have to go through it on your own, here is the outline of how we can do a proof by contradiction for a graph, specifically proving theorem 1.

Proof. First, suppose G is bipartite. Then its vertices can be partitioned into two sets V_1 and V_2, with every edge going between V_1 and V_2. Any cycle in G must alternate between vertices in V_1 and vertices in V_2. Therefore, after an odd number of edges, the cycle would end in the opposite set from where it started, so it cannot return to its starting vertex. Hence every cycle has even length.

Conversely, suppose G contains no cycles of odd length. Choose any connected component of G, and pick a vertex s. Put each vertex at even distance from s in V_1, and each vertex at odd distance from s in V_2. We claim this gives a valid bipartition.

Assume for contradiction that there is an edge (u, v) where u and v are in the same set. Then u and v have distances from s with the same parity. Taking shortest paths from s to u and from s to v, together with the edge (u, v), creates a cycle of odd length. This contradicts the assumption that G has no odd cycles.

Thus no edge connects two vertices in the same set, so each connected component is bipartite. Applying this construction to every connected component of G, the whole graph G is bipartite.

Therefore, G is bipartite if and only if it contains no cycles of odd length. □

1.7.2 Example 2 and 3. Cycle Detection and Leaves

Exercise 4 **Cycle Detection**

Design pseudocode for an algorithm which takes as an input an undirected graph, G, and an edge, $e = (u, v)$, and determines whether G has a cycle containing e.

Working Space

Answer on Page 18

Exercise 5 Leaf Removal

Proof the following statement: In any connected undirected graph, there is a vertex whose removal leaves the graph connected.

Working Space

Answer on Page 18

Sometimes, we may need to create new graphs as counterexamples to disprove a statement. Consider the following:

Theorem 2. *An graph* G *is strongly connected if every vertex is reachable from every other vertex.*

Another way to say this is that a select vertex of the graph has a path to every other vertex on the graph. Often, this comes up in finding *subgraphs* of graphs that are strongly connected, called *strongly connected components*.

Exercise 6 Strongly Connected Graphs

Create an example of a strongly connected directed graph $G = (V, E)$ such that, for every $v \in V$, removing v from G leaves a directed graph that is not strongly connected.

Hint: the simplest answer is usually the best. Start with one vertex.

Working Space

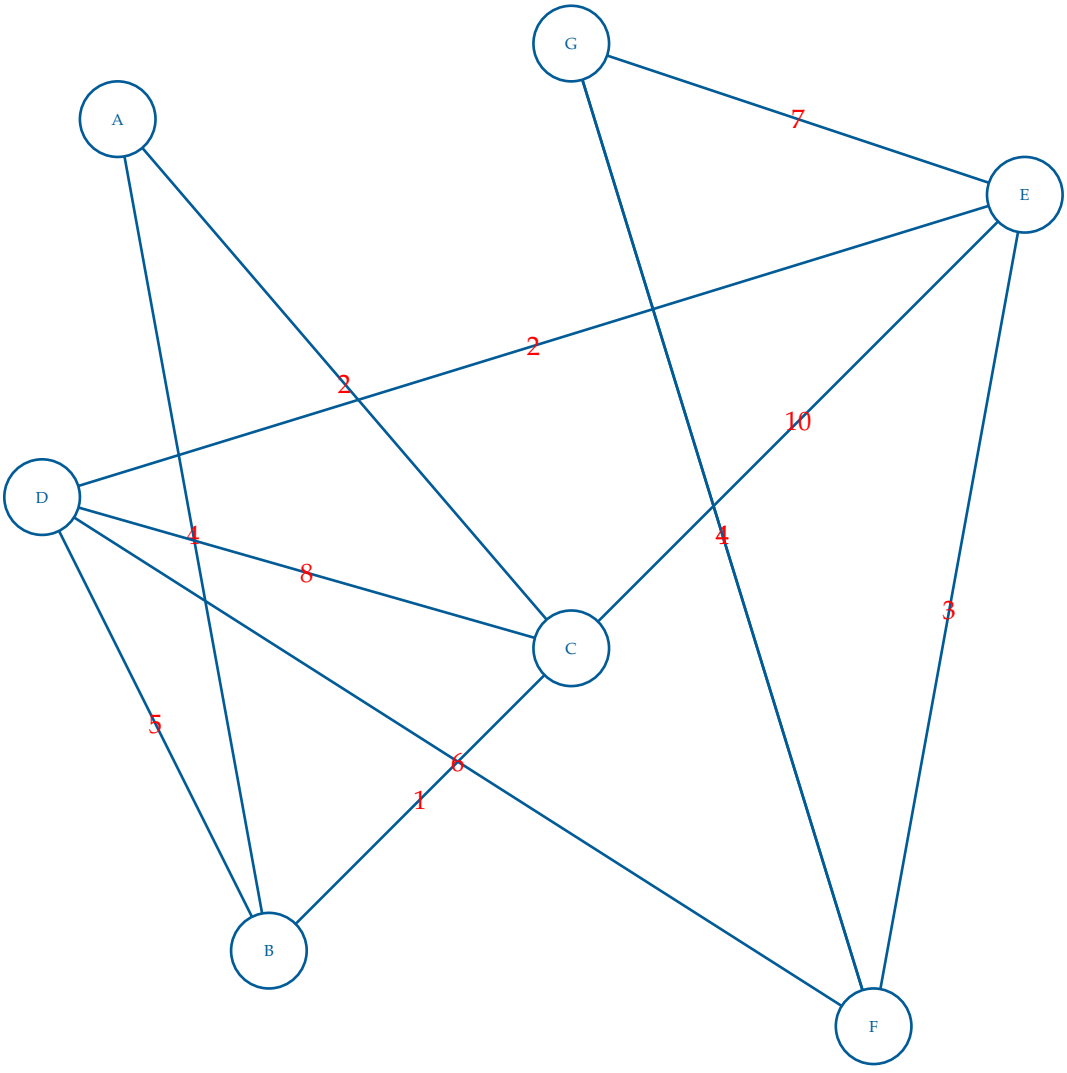
Answer on Page 19

org/) for more details.

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 3)



Answer to Exercise 2 (on page 7)

1. Injective, but not surjective. No two inputs map to the same output, but d is never

reached.

2. Surjective, but not injective. Every element of the codomain is reached, but both 1 and 4 map to a.

3. Bijective. Every element of the codomain is reached exactly once.

Answer to Exercise 3 (on page 13)

Assuming `FindOdd` exists, we can make the following pseudocode.

`IsBipartite`(G)
Require: G, a graph on which DFS has been run
Ensure: **true** if G is bipartite, **false** otherwise
1: **for** all back edges e **do**
2: **if** FindOdd$(G, e) =$ **true then**
3: **return false**
4: **end if**
5: **end for**
6: **return true**

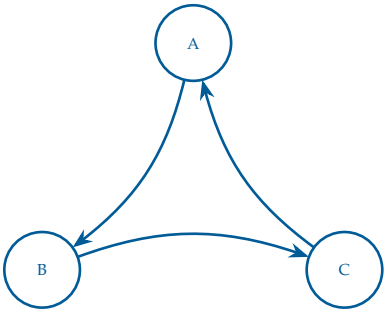
Answer to Exercise 4 (on page 14)

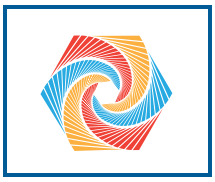
Remove the edge $e = (u, v)$ from G.
Run BFS or DFS starting from u.
if v is reached **then**
Return True.
else
Return False.
end if

Answer to Exercise 5 (on page 15)

Proof. Run DFS on given graph G. Find a leaf, l in the output DFS tree (sometimes called the explore tree). Remove l from the graph. Run DFS on the graph ($G' = G/\{l\}$). If the DFS tree is connected, then we are done. If not, then there is a vertex v in the DFS tree that is not connected to the rest of the graph. This means that v was only connected to l in the original graph. But this contradicts the fact that l was a leaf in the DFS tree, since v would have been explored before l. Since a leaf must exist, removing it from the explore tree does not create a new tree. Therefore, there must be a vertex whose removal leaves the graph connected. □

Answer to Exercise 6 (on page 15)





Index

A* Algorithm, 4
A-star Algorithm, 4
adjacency matrix, 2
algorithmic reduction, 9

bijection, 6
Bipartite Graph, 13

Chebyshev distance, 5
cycle, 11
Cycle Error, 11

Directed Acyclic Graph, 11
directed adjacency matrix, 2

Euclidean distance, 5

graph isomorphism, 7

heuristic, 4

injection, 6
Isomorphism, 7

matrix, 2

strongly connected, 15
strongly connected components, 15
surjection, 6

taxicab distance, 5
topological order, 10
Topological Sorting, 10